

GPIO Interrupt Controller

Learning, Debugging, Waveform Walkthrough, and Build History

SystemVerilog + APB | Vivado 2025.2 | XSim | 8 GPIO pins

Purpose: preserve the reasoning behind the working class-based verification environment, explain what each file does, document the errors encountered, and provide a repeatable way to study the waveform.

1. The final result

The final class-based verification environment successfully exercised GPIO reads/writes and a complete rising-edge interrupt lifecycle. The final XSim run produced the following checks:

```
PASS: READ DIR = 0x000000ff
PASS: READ DATA_OUT = 0x000000a5
INFO: Interrupt configured
PASS: INT_STATUS = 1
PASS: IRQ = 1
PASS: INT_STATUS CLEARED
PASS: IRQ CLEARED
```

The simulation completed normally at approximately 370 ns.

2. Big-picture architecture

Think of the design as four hardware blocks plus a class-based verification environment.

APB / software	->	gpio_registers	->	gpio_interrupt	->	irq
			->	gpio_io	->	GPIO pins

Signal flow: APB configuration -> register state -> interrupt detection / GPIO I/O -> IRQ back toward the system.

3. What each verification file does

File	Role
gpio_interface.sv	Collects APB, GPIO, reset, and IRQ signals into one virtual interface.
gpio_transaction.sv	Represents an APB transaction and carries GPIO-input stimulus information.
gpio_generator.sv	Creates the ordered test sequence and puts transactions into gen2drv.
gpio_driver.sv	Converts transactions into APB setup/access cycles or GPIO input changes.
gpio_monitor.sv	Observes the bus and DUT-facing results after the active clock edge.
gpio_scoreboard.sv	Compares observations against expected GPIO and interrupt behavior.
tb_gpio_controller.sv	Instantiates the DUT/environment, clock, reset, and runs all classes together.

4. APB: what the driver is doing

The driver performs a normal APB setup/access sequence. Setup establishes PSEL, address, direction, and data. Access asserts PENABLE. PREADY indicates completion, then the driver returns the bus to idle.

```
SETUP: PSEL=1, PENABLE=0 -> ACCESS: PSEL=1, PENABLE=1 -> WAIT PREADY -> IDLE
```

Why this matters: the generator should describe what the CPU wants to do; the driver owns bus timing.

5. Interrupt logic in plain English

The tested configuration is intentionally simple: GPIO0 is an input; interrupt type is edge; polarity selects a rising edge; interrupt enable allows the event to propagate. The external stimulus changes GPIO0 from 0 to 1. The interrupt block remembers the event in INT_STATUS and asserts IRQ. Software then writes 1 to INT_CLEAR and the interrupt returns to idle.

Memory trick: DATA = value, DIR = direction, TYPE/POLARITY = what counts, ENABLE = permission, STATUS = what happened, CLEAR = erase it.

6. The exact interrupt sequence

1. DIR = 0x00 -> GPIO0 is treated as an input.
2. INT_TYPE = 0x01 -> GPIO0 uses edge detection.
3. INT_POLARITY = 0x01 -> rising edge.
4. INT_ENABLE = 0x01 -> enable GPIO0 interrupt.
5. gpio_in changes 0x00 -> 0x01.
6. INT_STATUS[0] becomes 1.
7. IRQ becomes 1.
8. INT_CLEAR[0] is written as 1.
9. INT_STATUS[0] returns to 0.
10. IRQ returns to 0.

7. The mistakes that happened - and what they taught us

Wrong simulation top

The first simulation launched `gpio_controller` directly instead of `tb_gpio_controller`. The DUT was running without the intended testbench driving its ports. Fix: set `tb_gpio_controller` as the behavioral simulation top.

Monitor scheduling race

The scoreboard once saw `DIR` as 0 even though the APB write carried FF. The monitor sampled at the same simulation time step as the DUT register update. Fix: sample one small delay after the active edge. Lesson: understand HDL event scheduling; a clocking block is the more scalable future solution.

Duplicated scoreboard syntax

Repeated editing caused duplicated `if/else` and `case` structure, leading to a syntax error near `else`. Fix: rebuild the class as one clean block. Lesson: when structure becomes corrupted, simplify rather than patch indefinitely.

Premature simulation finish

The initial testbench used `#100` after reset release at 20 ns, so `$finish` happened at 120 ns before the fourth transaction finished. Fix: extend runtime; the final interrupt sequence completed around 370 ns. Better future practice: end on a transaction-done event rather than a fixed delay.

Interrupt not visible in first waveform view

The waveform was zoomed into only about 54-60 ns, so the later GPIO event was outside view. Fix: Zoom Fit first, then zoom into the 0->1 GPIO transition.

Vivado board warnings

Many board-part messages appeared during project startup. They did not block XSim behavioral simulation and were unrelated to DUT functionality.

Timescale warnings

Several source files did not declare an explicit timescale. XSim still used a 1 ps resolution and the test completed. Optional cleanup: add ``timescale 1ns/1ps` consistently.

Class-based bench was almost replaced

A compact single-module bench was considered to save time, but that would discard the `generator/driver/monitor/scoreboard` structure already built. We kept and extended the classes. Lesson: optimize the workflow, not away the architecture you need to demonstrate.

8. How to read the waveform

Start with Zoom Fit. Then read the wave from left to right. Watch PADDR/PWDATA/PWRITE to identify each APB access. Use gpio_in to find the external event and irq to see the propagated interrupt. For deeper understanding, expand the DUT hierarchy and add internal interrupt registers/signals when visible.

Basic GPIO example

PADDR=08, PWRITE=1, PWDATA=FF -> DIR becomes FF -> gpio_oe becomes FF.

PADDR=04, PWRITE=1, PWDATA=A5 -> DATA_OUT becomes A5 -> gpio_out becomes A5.

Interrupt example

Configuration writes appear at addresses 0C, 10, and 14 with data 01. Later gpio_in changes from 00 to 01. That 0->1 transition is the rising edge. IRQ then asserts. A write to address 1C with data 01 clears the pending event; IRQ deasserts.

9. Final waveform evidence

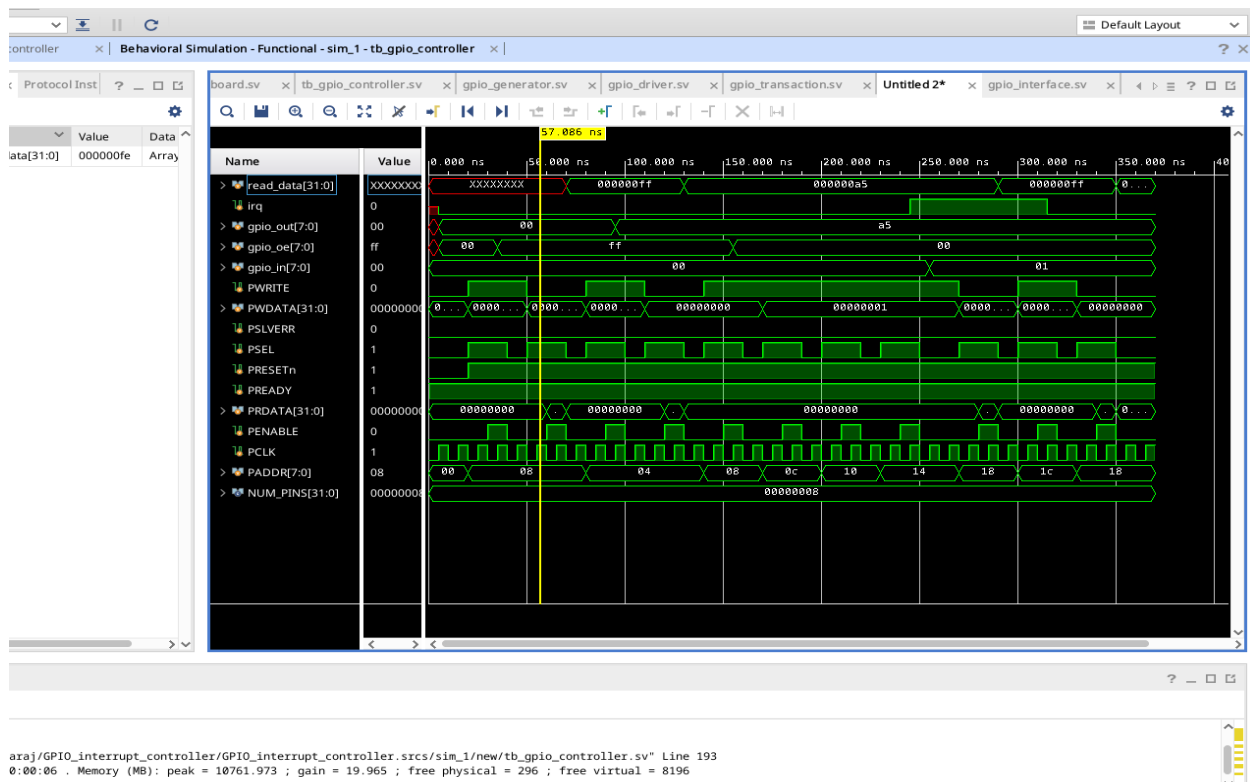


Figure: full final XSim waveform with the APB activity, GPIO input transition, and interrupt response visible.

10. Viva-ready explanation

"This project implements an 8-bit GPIO controller with an APB interface. Software accesses memory-mapped GPIO and interrupt registers. The direction register controls whether a pin is driven or treated as an input. The interrupt block detects a configured edge or level condition, latches the event into status, and asserts an aggregated IRQ when enabled. The verification environment is class-based: the generator creates transactions, the driver applies APB timing or GPIO stimulus, the monitor observes the DUT, and the scoreboard checks the result. The final simulation verifies GPIO read/write behavior and a complete rising-edge interrupt lifecycle."